

Лабораторная работа №6

Решение моделей в непрерывном и дискретном времени

Дурдалыев Максат

Содержание

1	Введение	5
1.1	Цели и задачи	5
2	Выполнение лабораторной работы	6
2.1	Решение обыкновенных дифференциальных уравнений	6
2.2	Модель экспоненциального роста	6
2.3	Система Лоренца	11
2.4	Модель Лотки–Вольтерры	15
2.5	Самостоятельное выполнение	19
3	Выводы	47
	Список литературы	48

Список иллюстраций

2.1	График модели экспоненциального роста	7
2.2	График модели экспоненциального роста	8
2.3	График модели экспоненциального роста (задана точность решения)	10
2.4	Аттрактор Лоренца	12
2.5	Аттрактор Лоренца (интерполяция отключена)	14
2.6	Модель Лотки–Вольтерры: динамика изменения численности популяций	16
2.7	Модель Лотки–Вольтерры: фазовый портрет	18
2.8	Решение задания №1	20
2.9	Решение задания №1	21
2.10	Решение задания №2	23
2.11	Решение задания №2	24
2.12	Решение задания №3	26
2.13	Решение задания №3	27
2.14	Решение задания №3	28
2.15	Решение задания №4	30
2.16	Решение задания №4	31
2.17	Решение задания №5	33
2.18	Решение задания №5	34
2.19	Решение задания №5	35
2.20	Решение задания №6	37
2.21	Решение задания №6	38
2.22	Решение задания №6	39
2.23	Решение задания №7	41
2.24	Решение задания №7	42
2.25	Решение задания №7	43
2.26	Решение задания №8	45
2.27	Решение задания №8	46

Список таблиц

1 Введение

1.1 Цели и задачи

Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени[2].

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы[1].

2 Выполнение лабораторной работы

2.1 Решение обыкновенных дифференциальных уравнений

Вспомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u .

Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет `differentialEquations.jl`.

2.2 Модель экспоненциального роста

Рассмотрим пример использования этого пакета для решения уравнения модели экспоненциального роста, описываемую уравнением, где a — коэффициент роста.

Численное решение в Julia будет иметь следующий вид, а также график, соответствующий полученному решению (рис. [fig:001] - рис. [fig:002]):

```

# подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")

[ ] Julia

using DifferentialEquations

[2] ✓ 4.9s Julia

# задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0
# задаём интервал времени:
tspan = (0.0,1.0)

[4] ✓ 0.3s Julia

... (0.0, 1.0)

# решение:
prob = ODEProblem(f,u0,tspan)
sol = solve(prob)

[5] ✓ 3.4s Julia

... retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.10042494449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
 1.1034222047865465
 1.4121908713484919
 1.9730384457359198
 2.6644561424814266

```

Рисунок 2.1: График модели экспоненциального роста

```
# подключаем необходимые пакеты:
```

```
Pkg.add("Plots")
```

```
using Plots
```

[6] ✓ 8.7s

Julia

... Resolving package versions...

Project No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Proj`

Manifest No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Mani`

```
# строим графики:
```

```
plot(sol, linewidth=5, title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label
```

```
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

[7] ✓ 5.8s

Julia

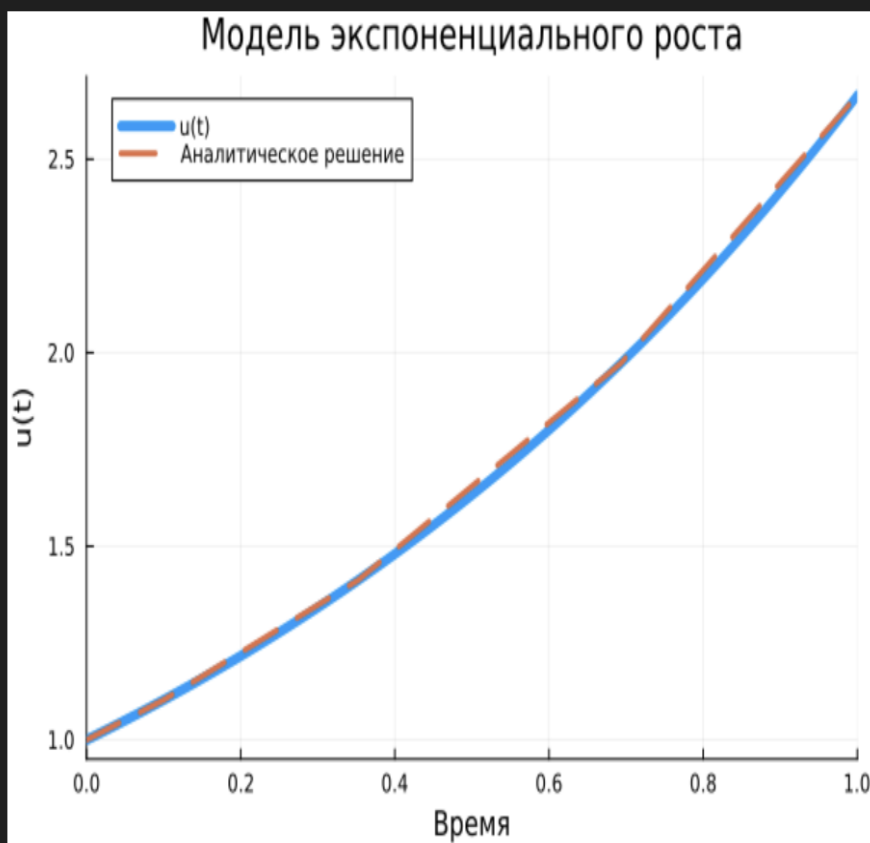


Рисунок 2.2: График модели экспоненциального роста

При построении одного из графиков использовался вызов `sol.t`, чтобы захватить массив моментов времени. Массив решений можно получить, воспользовавшись `sol.u`.

Если требуется задать точность решения, то можно воспользоваться параметрами `abstol` (задаёт близость к нулю) и `reltol` (задаёт относительную точность). По умолчанию эти параметры имеют значение `abstol = 1e-6` и `reltol = 1e-3`.

Для модели экспоненциального роста (рис. [fig:003]):

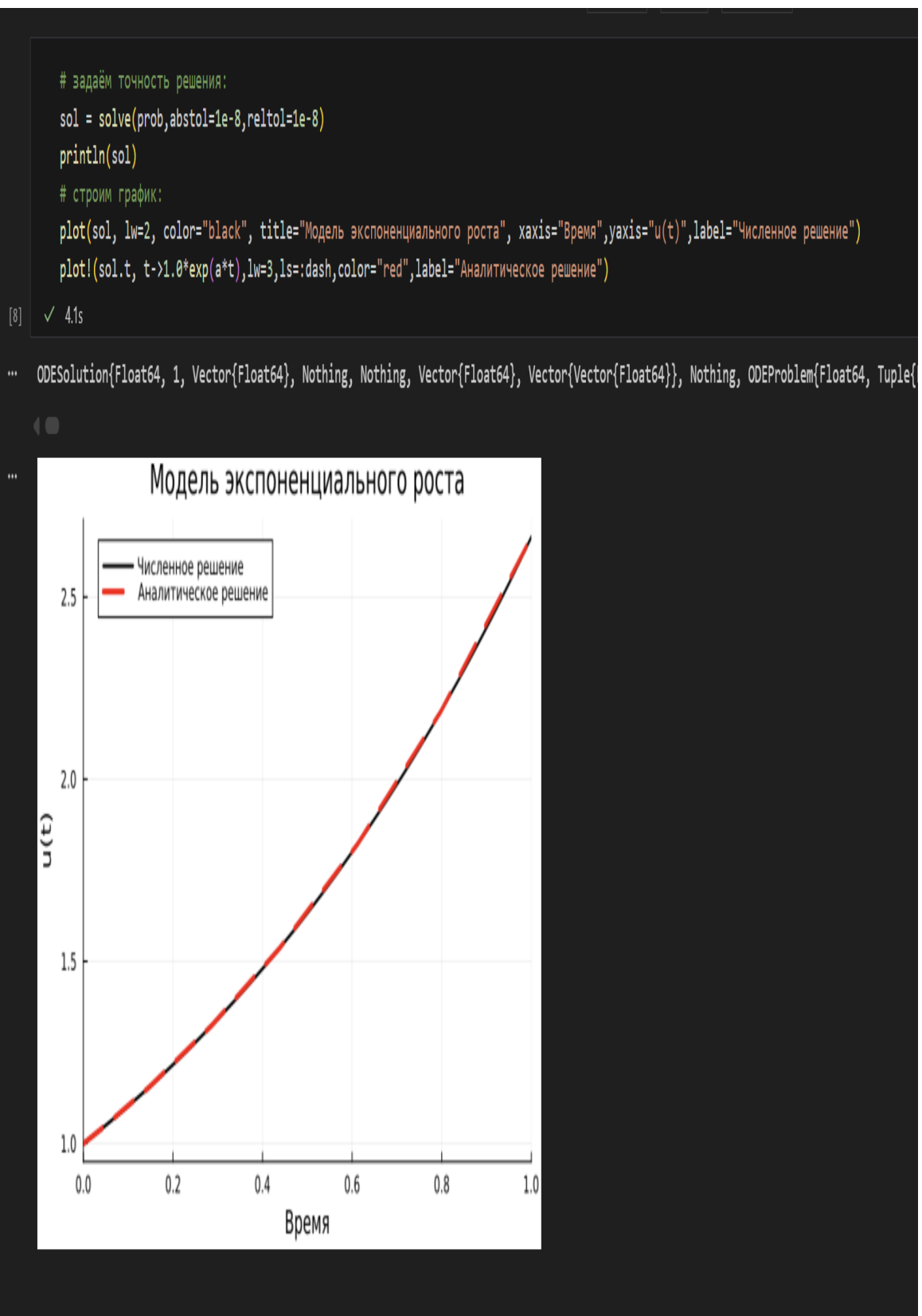


Рисунок 2.3: График модели экспоненциального роста (задана точность решения)

2.3 Система Лоренца

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка.

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующем усечением до первых-вторых гармоник.

Решение системы неустойчиво на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления.

Численное решение в Julia будет иметь следующий вид (рис. [fig:004]):

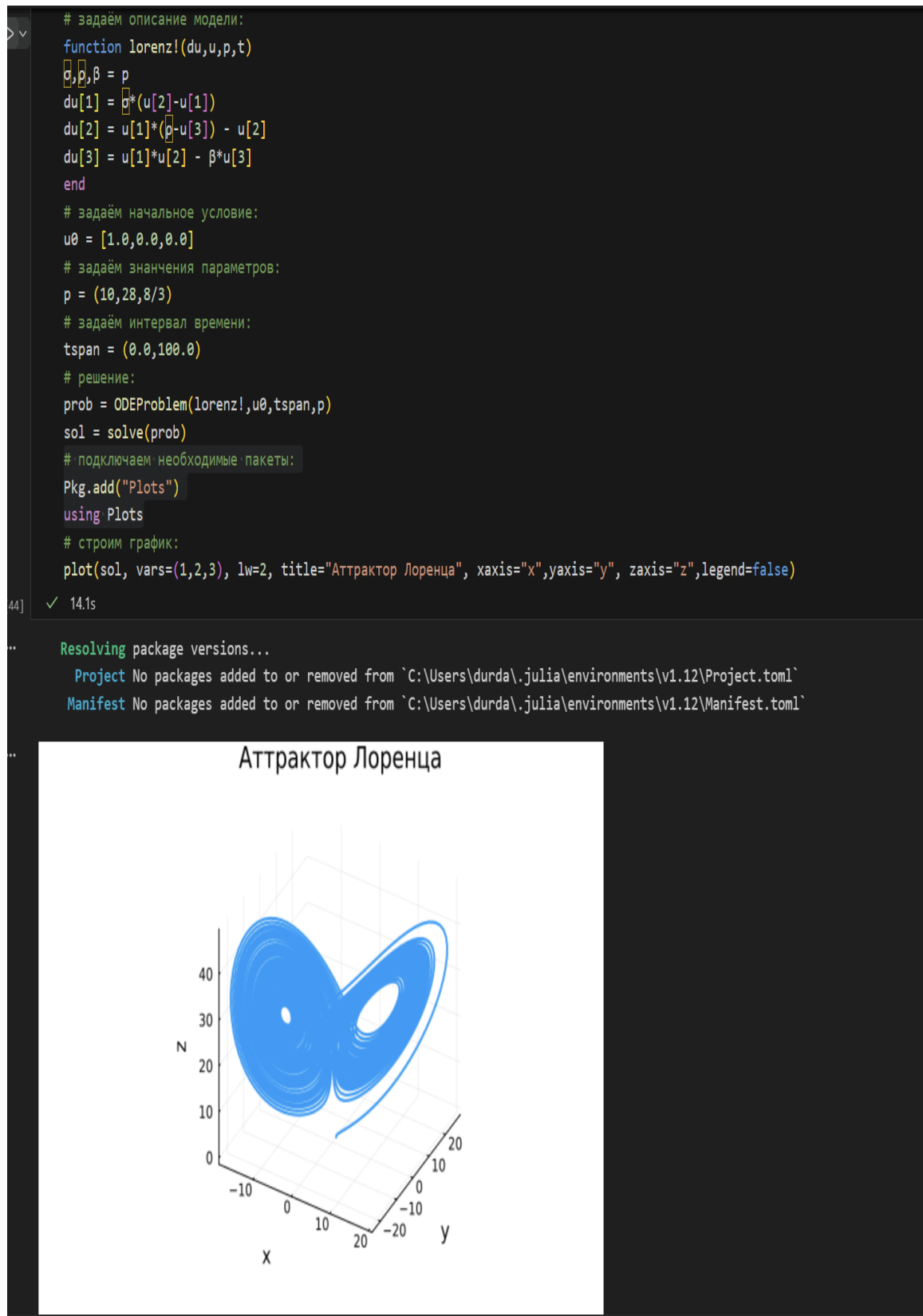


Рисунок 2.4: Аттрактор Лоренца

Можно отключить интерполяцию (рис. [fig:005]):



отключаем интерполяцию:

```
plot(sol,vars=(1,2,3),denseplot=false, lw=1, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)
```

[12]

✓ 0.1s

...

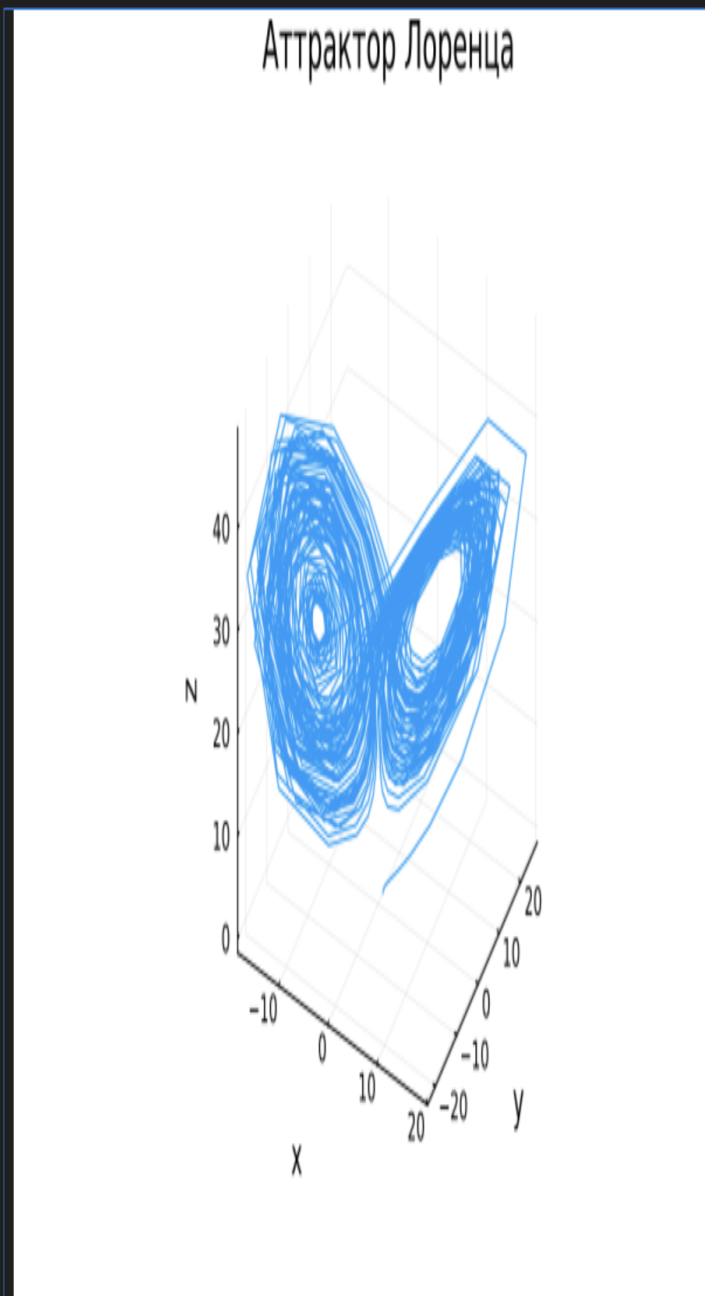


Рисунок 2.5: Аттрактор Лоренца (интерполяция отключена)

2.4 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух видов типа «хищник – жертва».

Численное решение в Julia будет иметь следующий вид (рис. [fig:006]):

```

# задаём описание модели:
lv! = @ode_def LotkaVolterra begin
  dx = a*x - b*x*y
  dy = -c*y + d*x*y
end a b c d

# задаём начальное условие:
u0 = [1.0,1.0]

# задаём значения параметров:
p = (1.5,1.0,3.0,1.0)

# задаём интервал времени:
tspan = (0.0,10.0)

# решение:
prob = ODEProblem(lv!,u0,tspan,p)
sol = solve(prob)
plot(sol, label = ["Жертвы" "Хищники"], color="black", ls=[:solid :dash], title="Модель Лотки - Вольтерры", xaxis="Время",yaxis="Размер популяции")

```

15] ✓ 5.1s

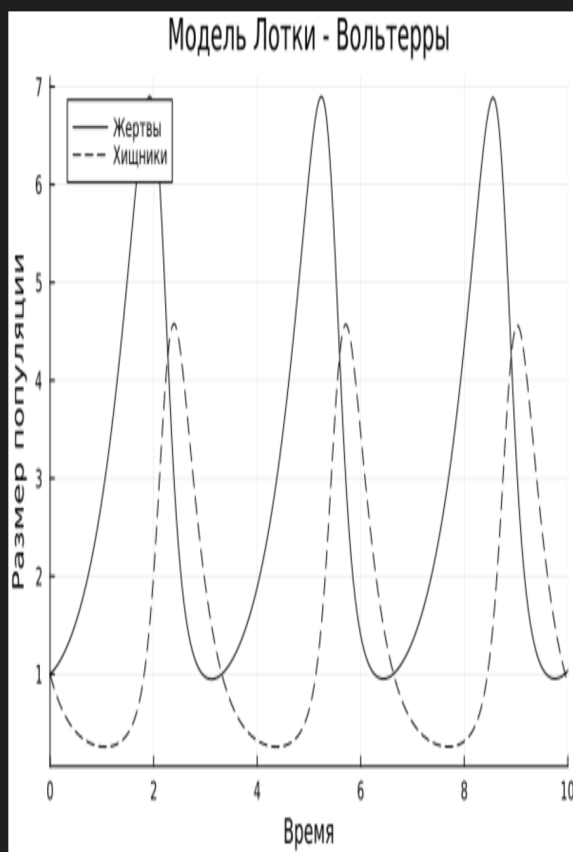


Рисунок 2.6: Модель Лотки–Вольтерры: динамика изменения численности популяций

Фазовый портрет (рис. [fig:007]):

```
# фазовый портрет:  
plot(sol,vars=(1,2), color="black", xaxis="Жертвы",yaxis="Хищники", legend=false)  
[16] ✓ 0.2s
```

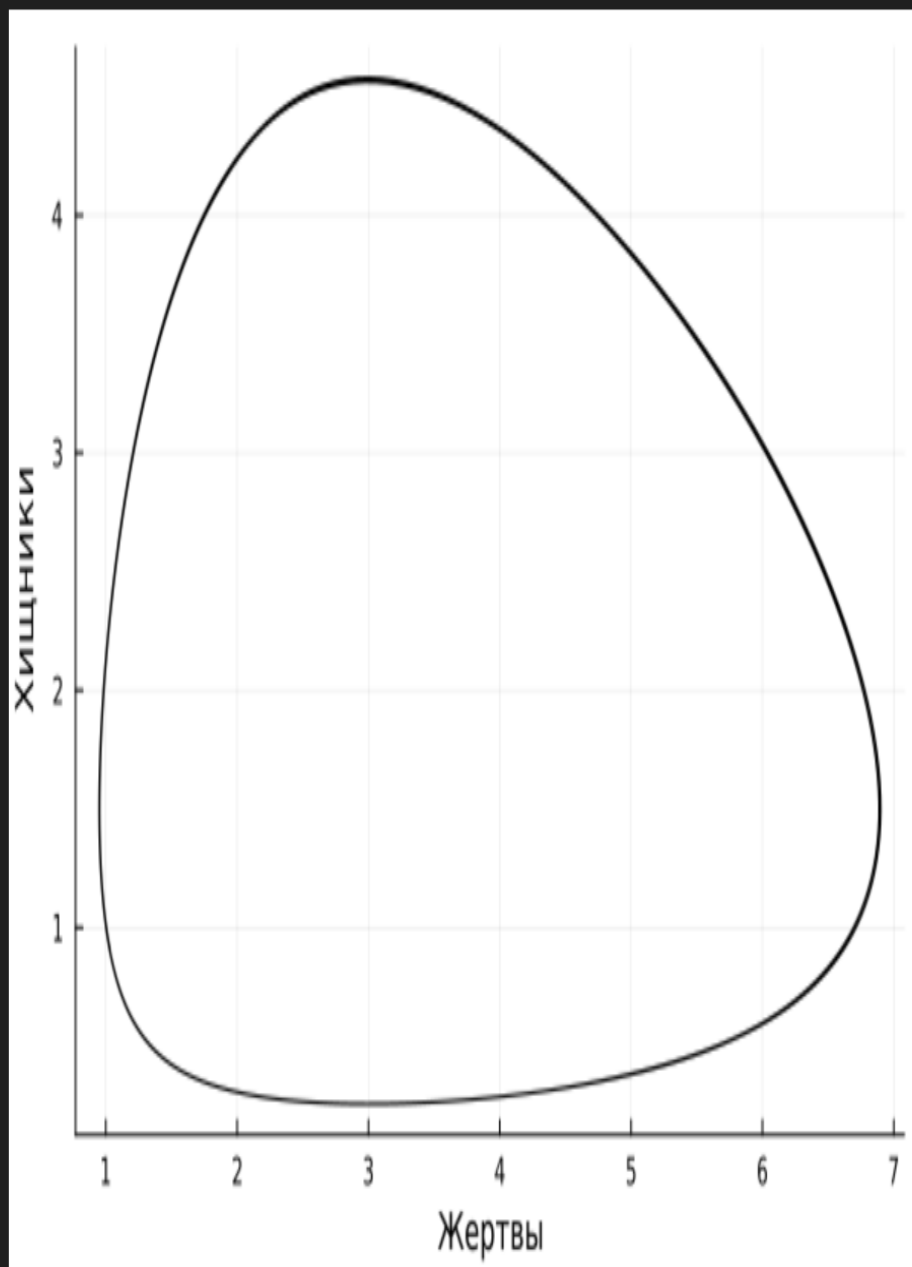


Рисунок 2.7: Модель Лотки–Вольтерры: фазовый портрет

2.5 Самостоятельное выполнение

Выполнение задания №1 (рис. [fig:008] - рис. [fig:009]):

Задания

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

Создать + Code + Markdown

```
f(u, p, t) = a * u
b = 0.5
c = 0.1
a = b - c
u0 = 1.0
tspan = (0.0, 10.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Модель Мальтуса", label="Численное решение", color="pink", linewidth = 3)
```

[17]

✓ 1.8s

Julia

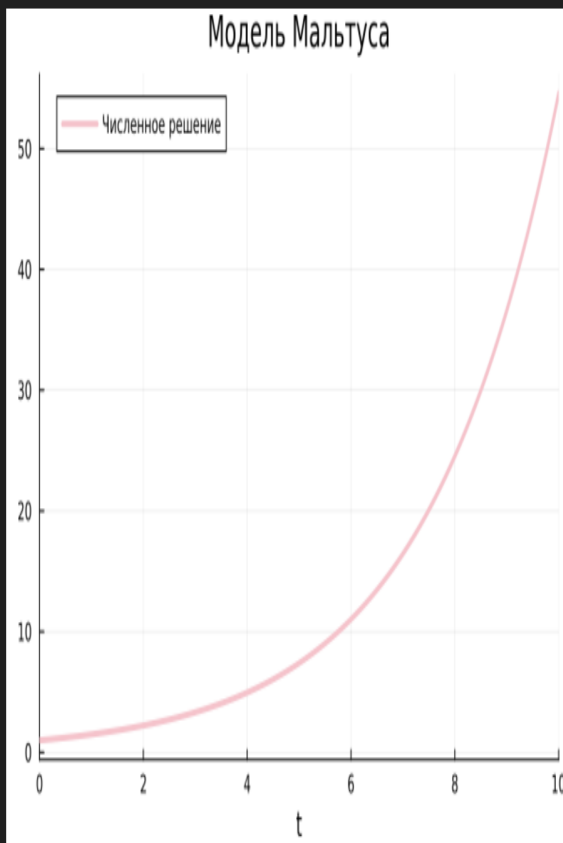


Рисунок 2.8: Решение задания №1

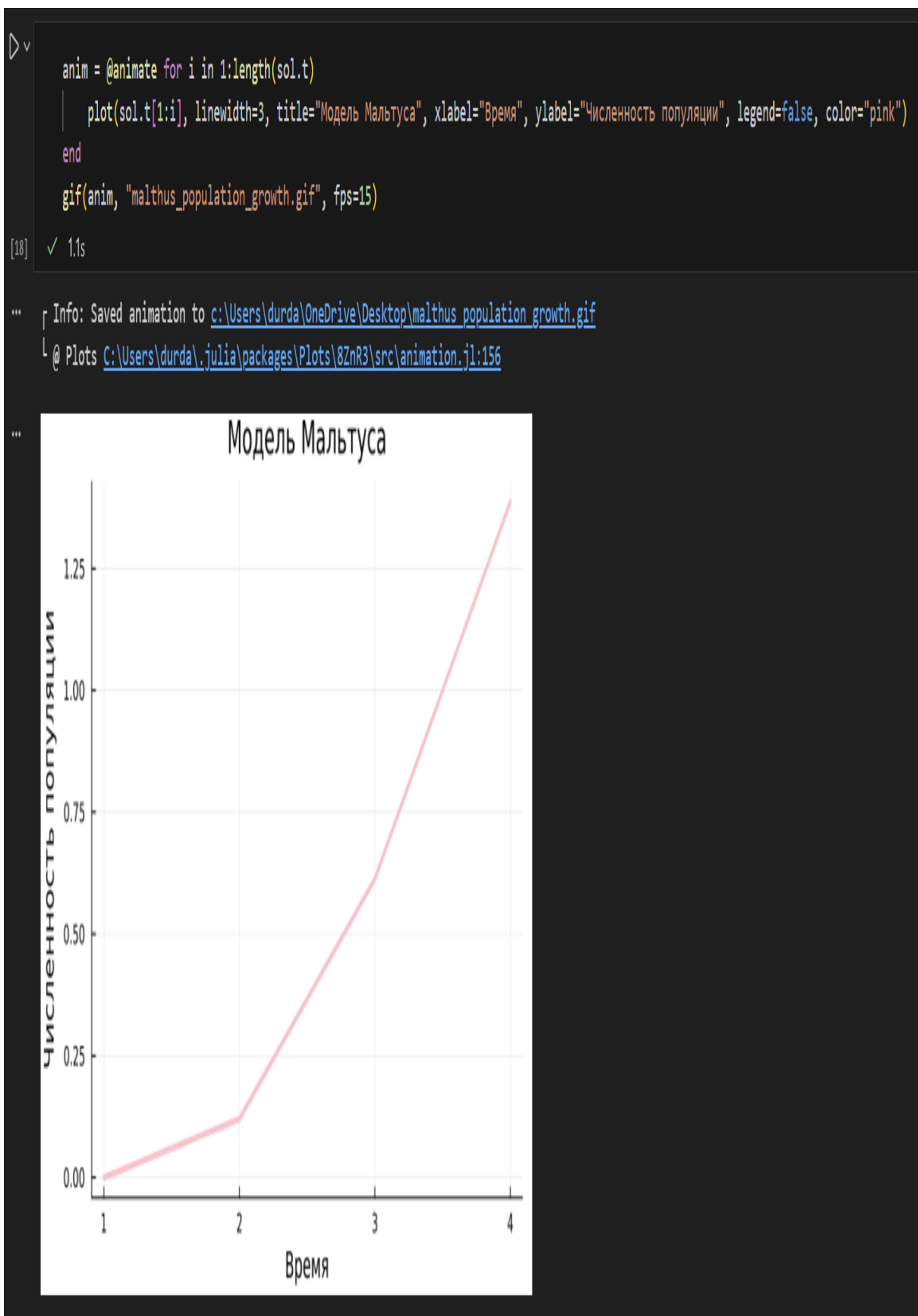


Рисунок 2.9: Решение задания №1

Выполнение задания №2 (рис. [fig:010] - рис. [fig:011]):

2. Реализовать и проанализировать логистическую модель роста популяции. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
f(u, p, t) = r * u * (1 - u/k)
r = 0.5
k = 100
u0 = 1.0
tspan = (0.0, 20.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Логистическая модель роста популяции", label="Численность популяции", color="green", linewidth = 3)
```

22] ✓ 1.4s

Julia

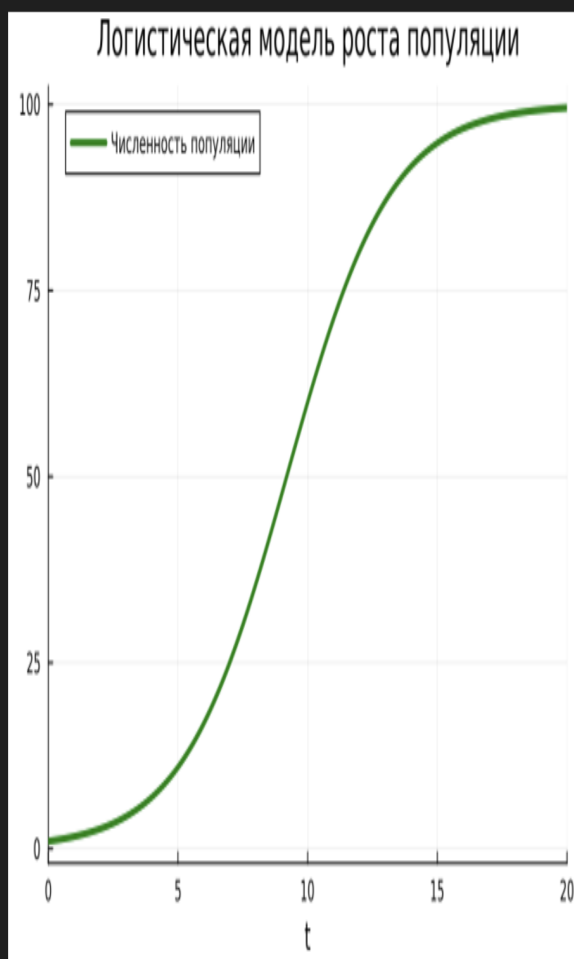


Рисунок 2.10: Решение задания №2

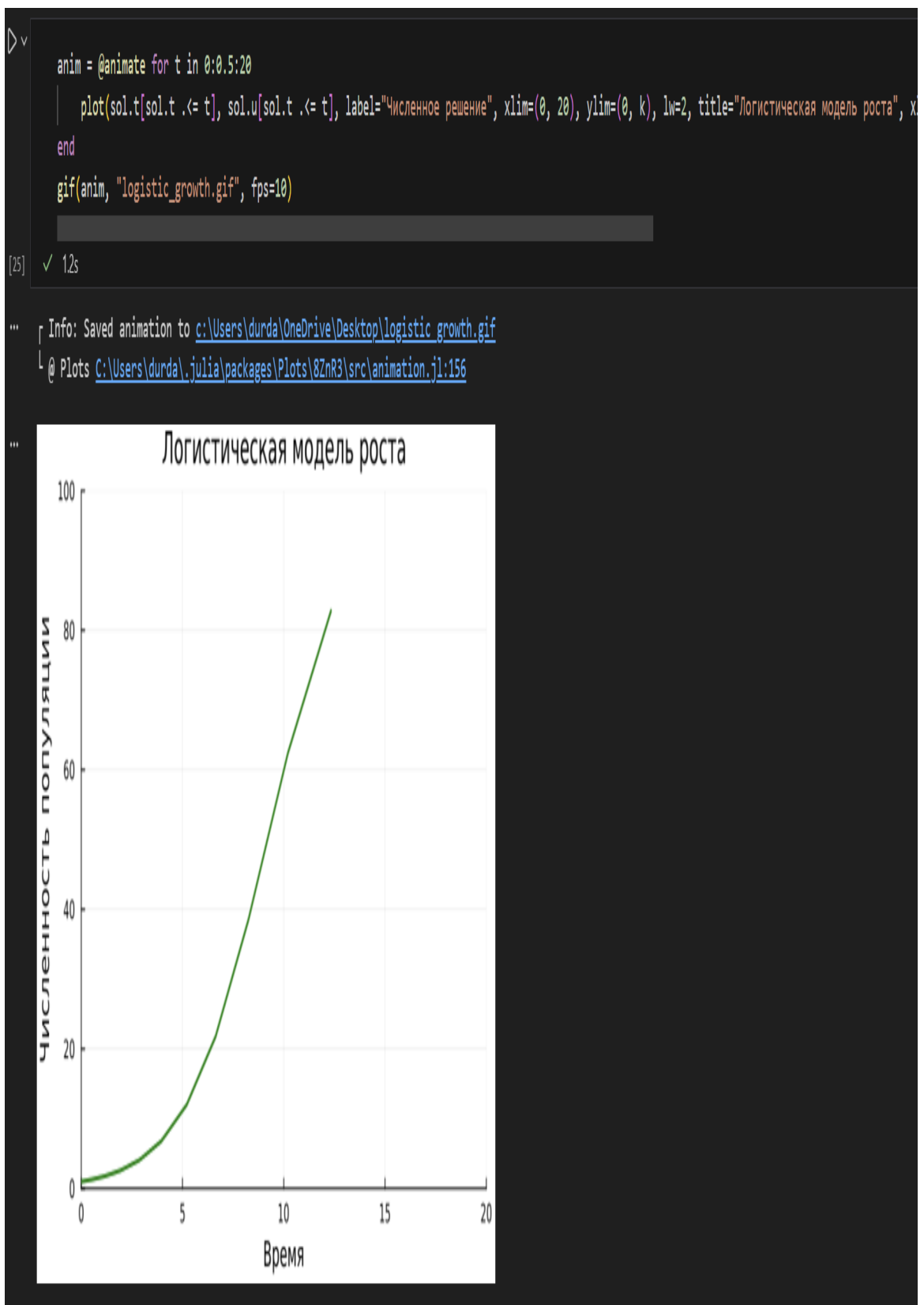


Рисунок 2.11: Решение задания №2

Выполнение задания №3 (рис. [fig:012] - рис. [fig:014]):

3. Реализовать и проанализировать модель эпидемии Кермака-Маккендрика (SIR-модель). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
function sir(du, u, p, t)
    s, i, r = u
    β, γ = p
    du[1] = -β * s * i # ds/dt
    du[2] = β * s * i - γ * i # di/dt
    du[3] = γ * i # dr/dt
end
```

26] ✓ 0.0s

Julia

... sir (generic function with 1 method)

Рисунок 2.12: Решение задания №3

```

# Начальные данные
u0 = [0.95, 0.05, 0.0] # 95% восприимчивых, 5% инфицированных, 0% выздоровевших

# Параметры
β = 0.4 # Коэффициент передачи инфекции
γ = 0.15 # Коэффициент выздоровления

p = (β, γ)
tspan = (0.0, 100.0)

prob_sir = ODEProblem(sir, u0, tspan, p)
sol_sir = solve(prob_sir, Tsit5())

plot(sol_sir, title="SIR модель эпидемии", label=["S" "I" "R"], linewidth=2)
xlabel!("Время (дни)")
ylabel!("Доля населения")

```

[27] ✓ 1.7s

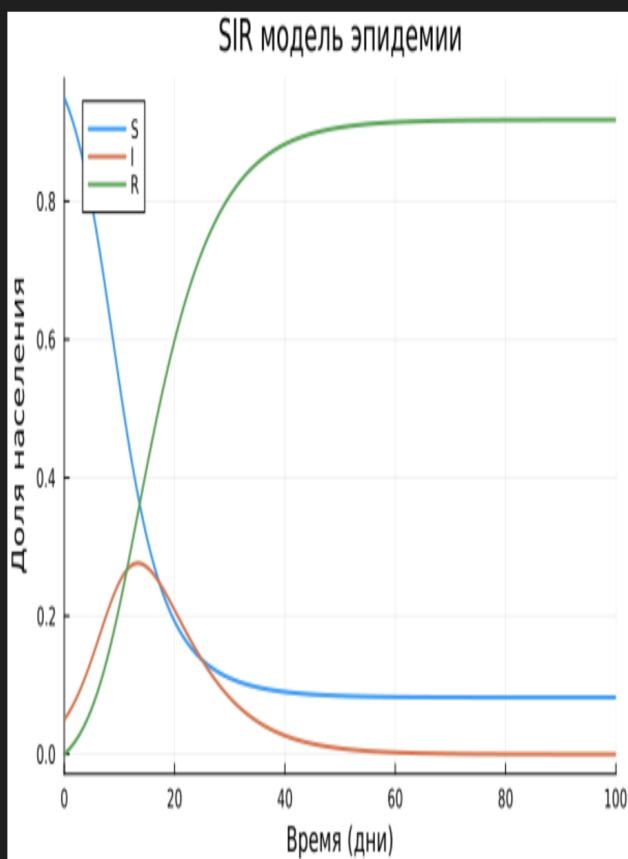


Рисунок 2.13: Решение задания №3

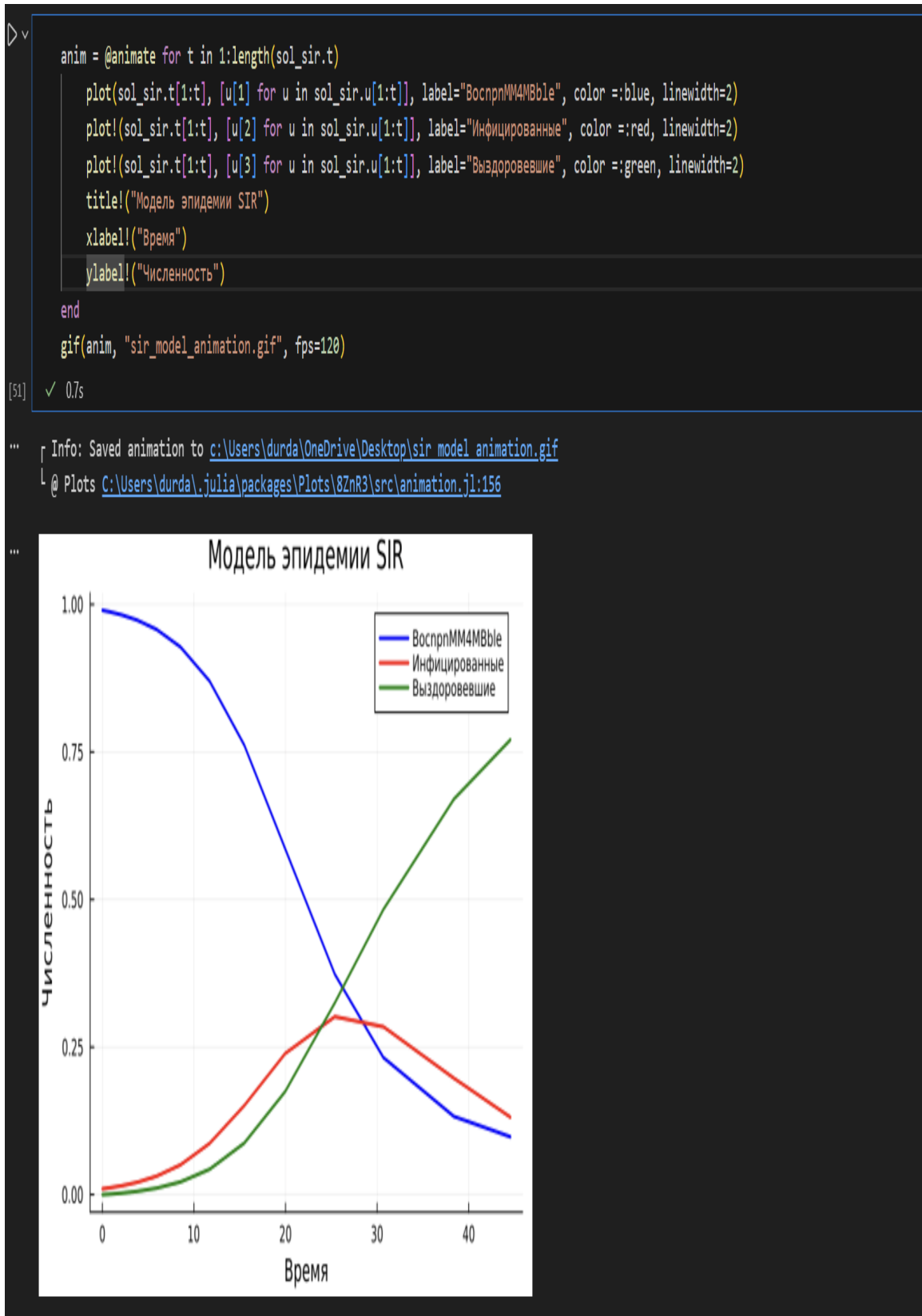


Рисунок 2.14: Решение задания №3

Выполнение задания №4 (рис. [fig:015] - рис. [fig:016]):

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed). Исследуйте, сравните с SIR.

```
function seir(du, u, p, t)
    s, e, i, r = u
     $\beta$ ,  $\sigma$ ,  $\gamma$  = p
    N = s + e + i + r # Общая численность населения

    du[1] = - $\beta$  * s * i / N # ds/dt: Восприимчивые
    du[2] =  $\beta$  * s * i / N -  $\sigma$  * e # de/dt: Экспонированные
    du[3] =  $\sigma$  * e -  $\gamma$  * i # di/dt: Инфицированные
    du[4] =  $\gamma$  * i # dr/dt: Выздоровевшие

end
```

[29] ✓ 0.0s

""" seir (generic function with 1 method)

Рисунок 2.15: Решение задания №4

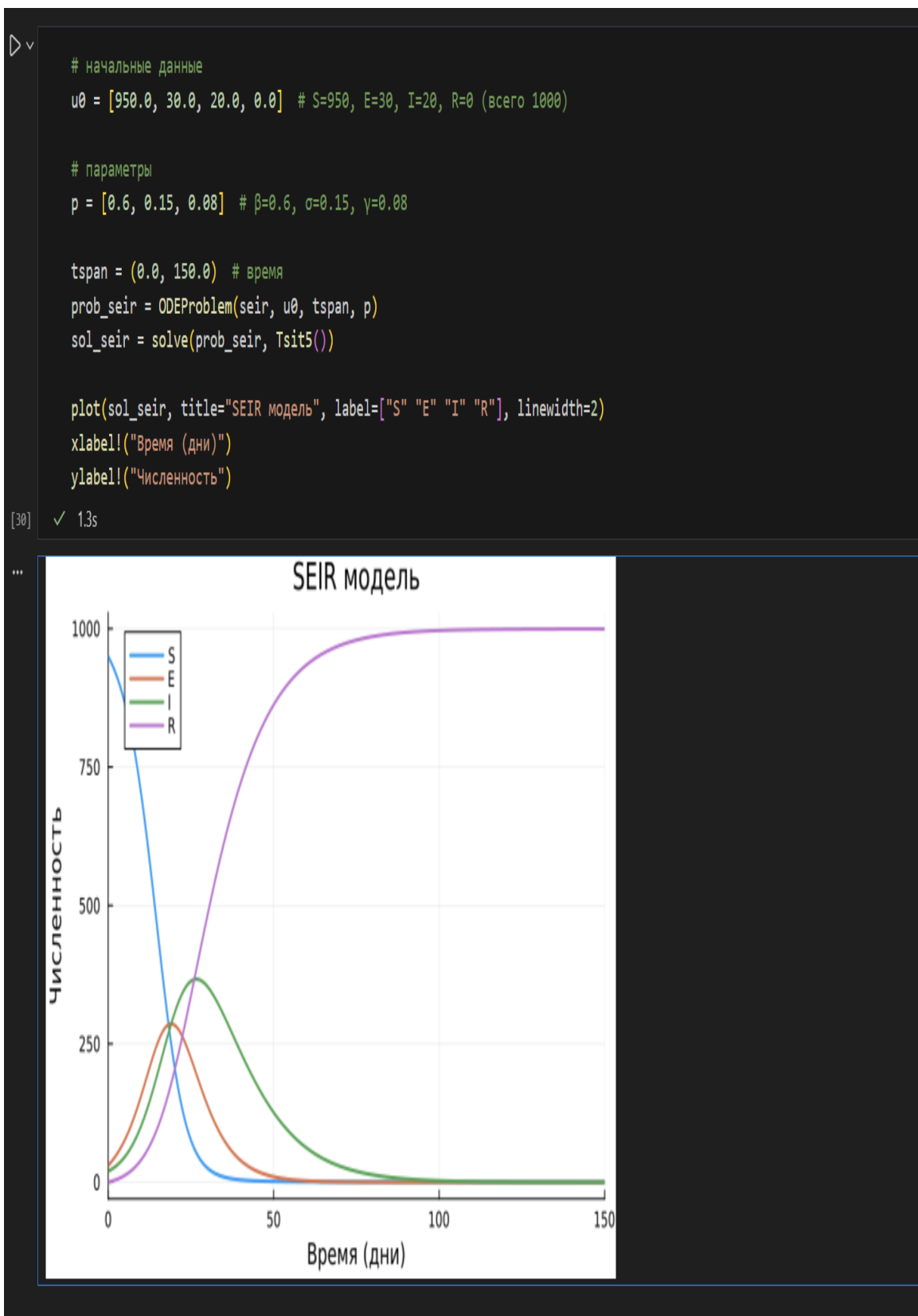


Рисунок 2.16: Решение задания №4

Выполнение задания №5 (рис. [fig:017] - рис. [fig:019]):

5. Для дискретной модели Лотки-Вольтерры найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

```
a = 2
c = 1
d = 5
X0 = [0.5, 0.3]
tspan = 50
X1 = zeros(tspan)
X2 = zeros(tspan)
X1[1] = X0[1]
X2[1] = X0[2]
for t in 1:tspan-1
    X1[t+1] = a * X1[t] * (1-X1[t]) - X1[t] * X2[t]
    X2[t+1] = -c * X2[t] + d * X1[t] * X2[t]
end
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")
plot!(1:tspan, X2, label="Хищники(X2)")
```

4] ✓ 0.1s Julia

Рисунок 2.17: Решение задания №5

```
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")  
plot(1:tspan, X2, label="Хищники(X2)")
```

4] ✓ 0.1s

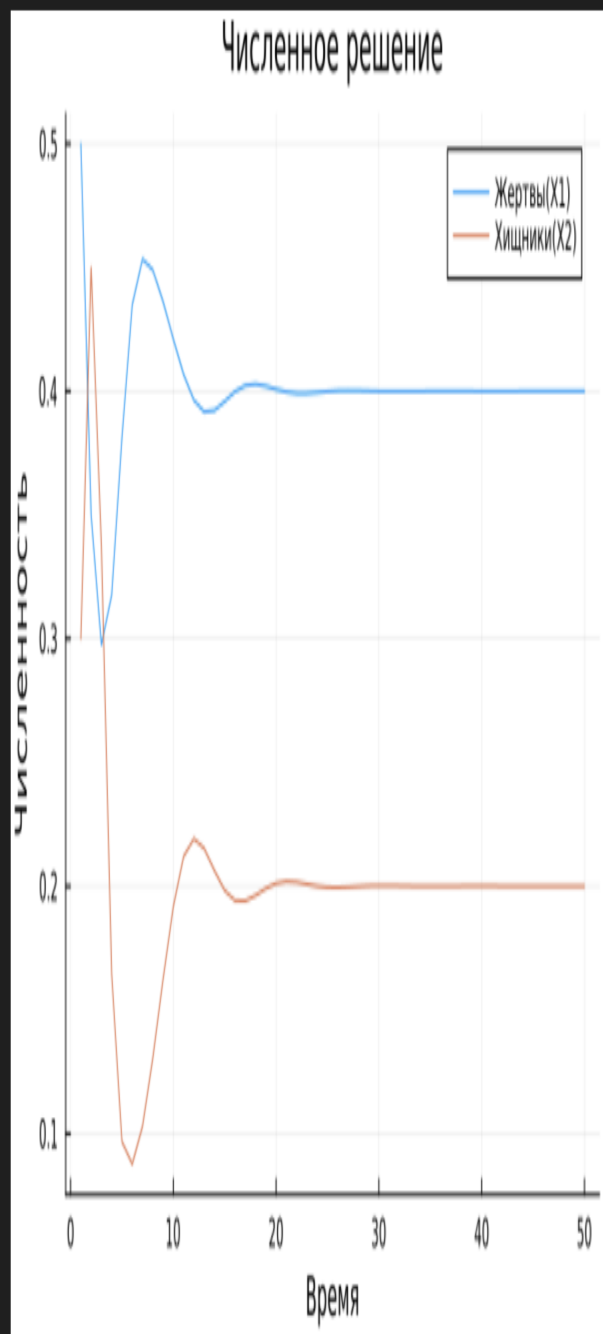


Рисунок 2.18: Решение задания №5

```
plot(X1, X2, xlabel="Жертвы(X1)", ylabel="Хищники(X2)", title="Фазовый портрет")
```

[35] ✓ 0.0s

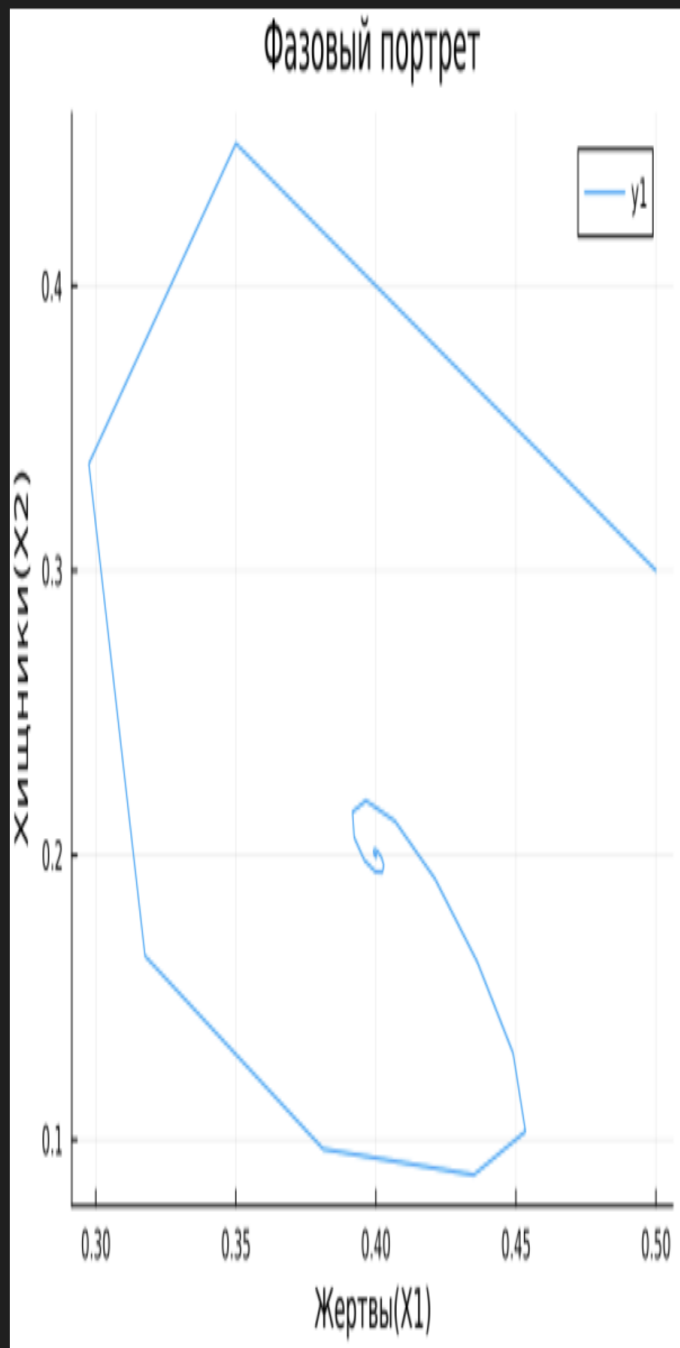


Рисунок 2.19: Решение задания №5

Выполнение задания №6 (рис. [fig:020] - рис. [fig:022]):

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
function competition(du, u, p, t)
    α, β = p
    x, y = u
    du[1] = α * x - β * x * y
    du[2] = α * y - β * x * y
end
```

[36] ✓ 0.0s

Julia

"" competition (generic function with 1 method)

Рисунок 2.20: Решение задания №6

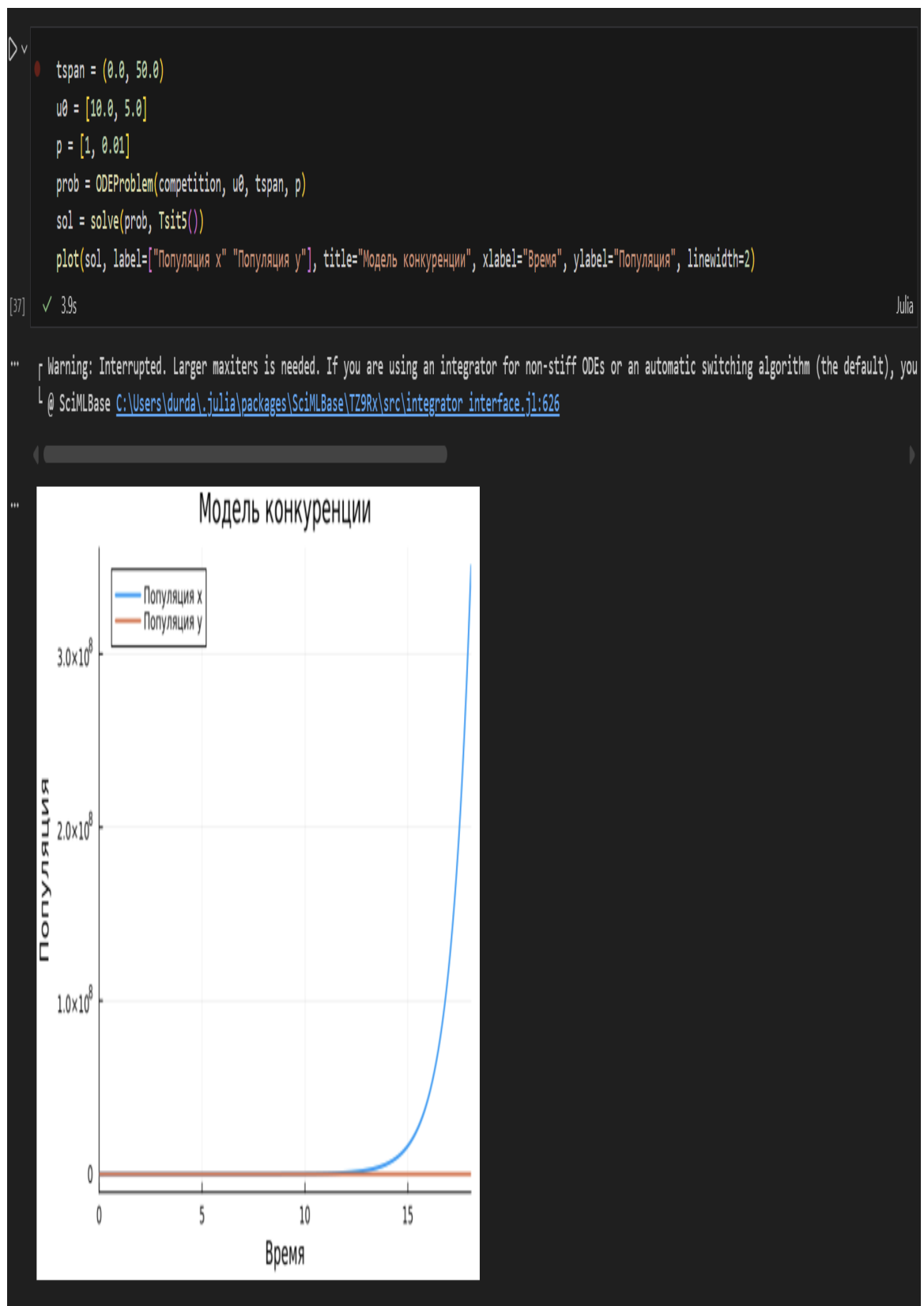


Рисунок 2.21: Решение задания №6

```
plot(sol, idxs=(1, 2), title="Конкурентные отношения")
```

[38] ✓ 0.6s

Julia

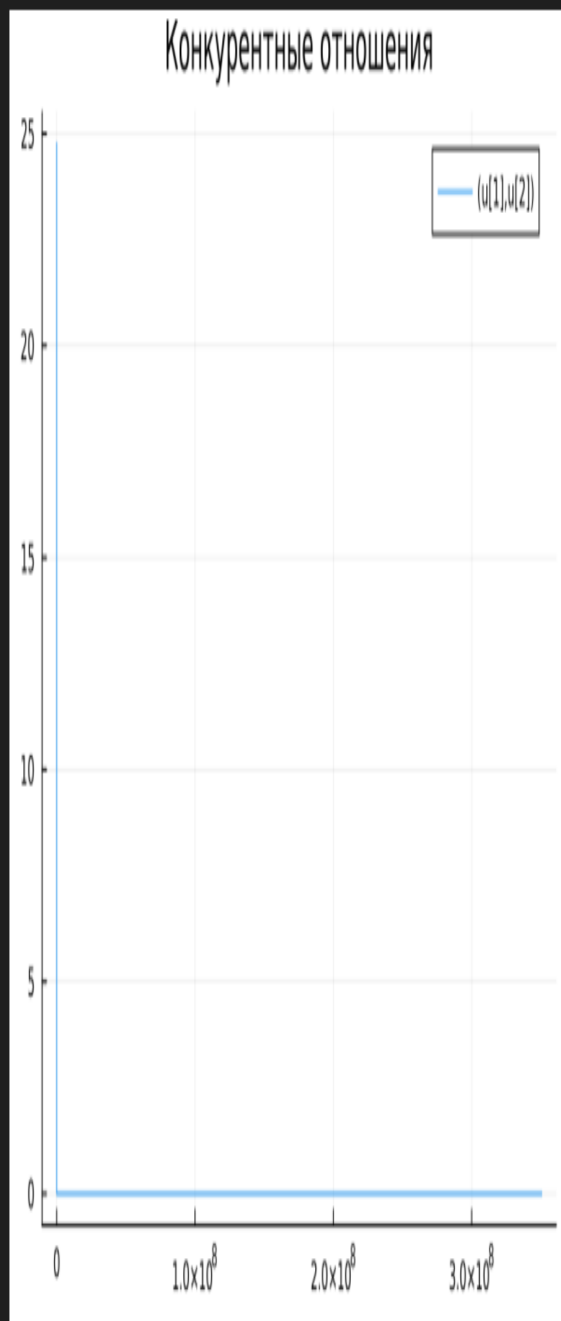
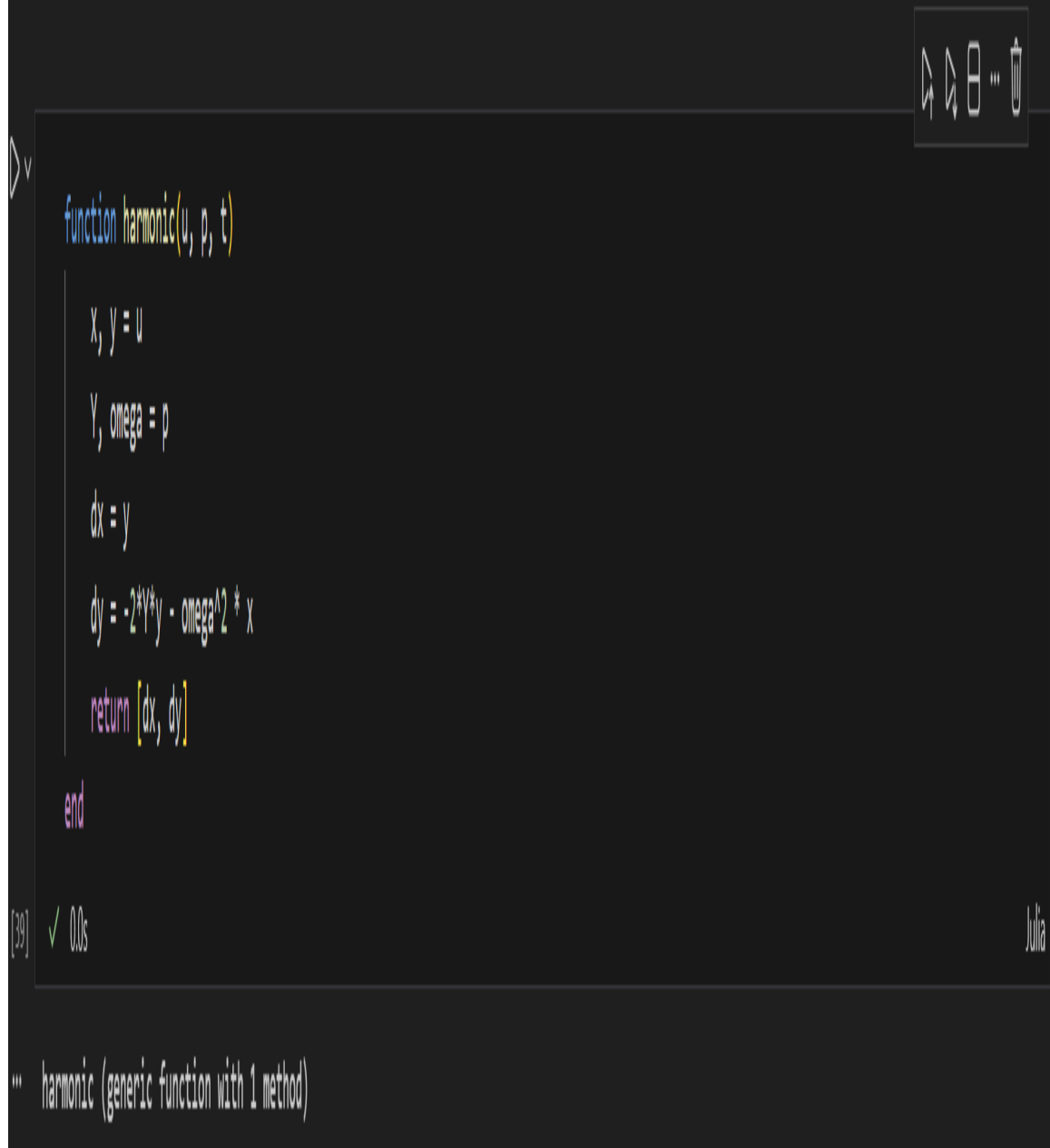


Рисунок 2.22: Решение задания №6

Выполнение задания №7 (рис. [fig:023] - рис. [fig:025]):

7. Реализовать на языке Julia модель консервативного гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.



```
function harmonic(u, p, t)
    x, y = u
    Y, omega = p
    dx = y
    dy = -2*Y*y - omega^2 * x
    return [dx, dy]
end

[39] ✓ 0.0s Julia
```

... harmonic (generic function with 1 method)

Рисунок 2.23: Решение задания №7

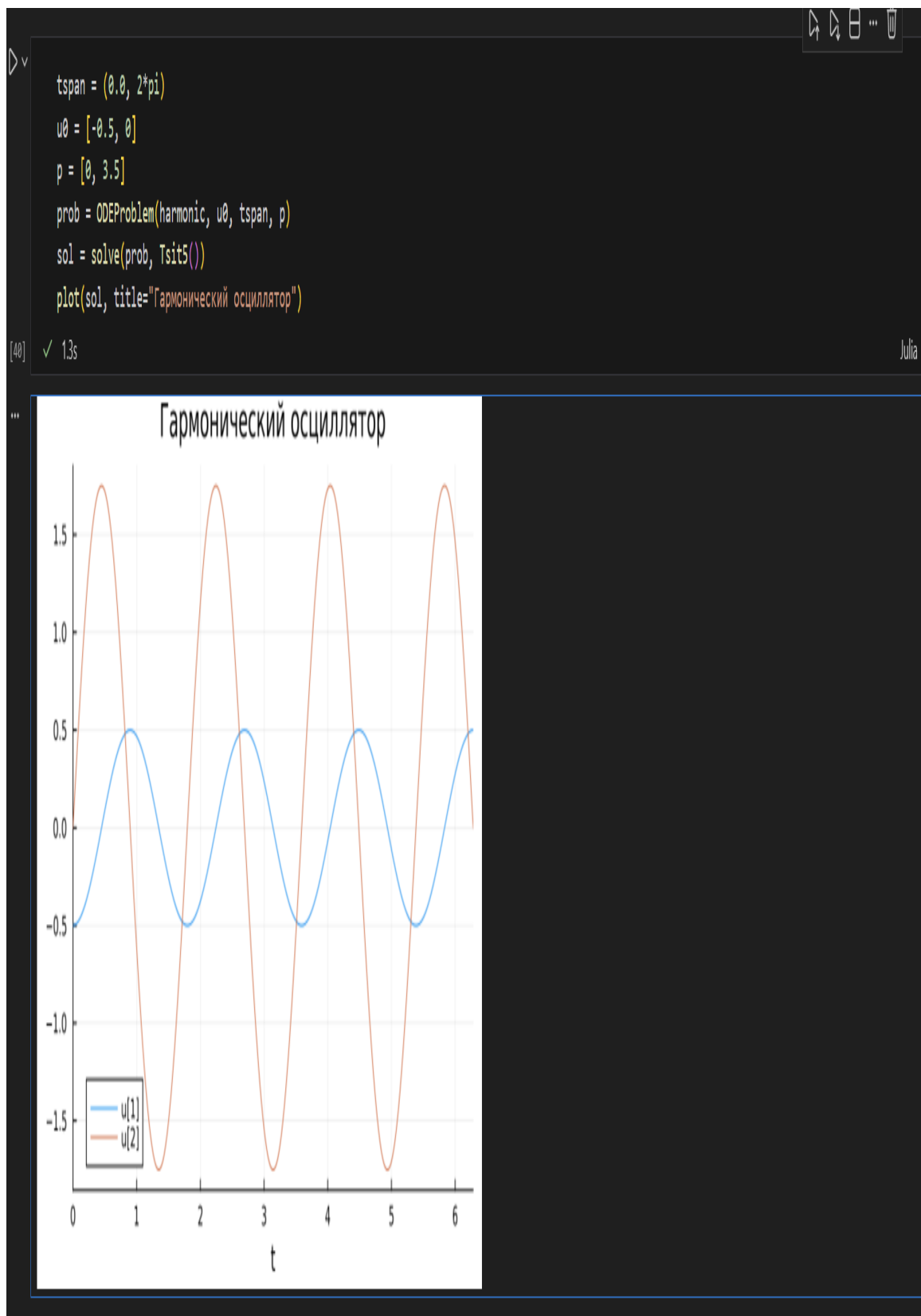


Рисунок 2.24: Решение задания №7

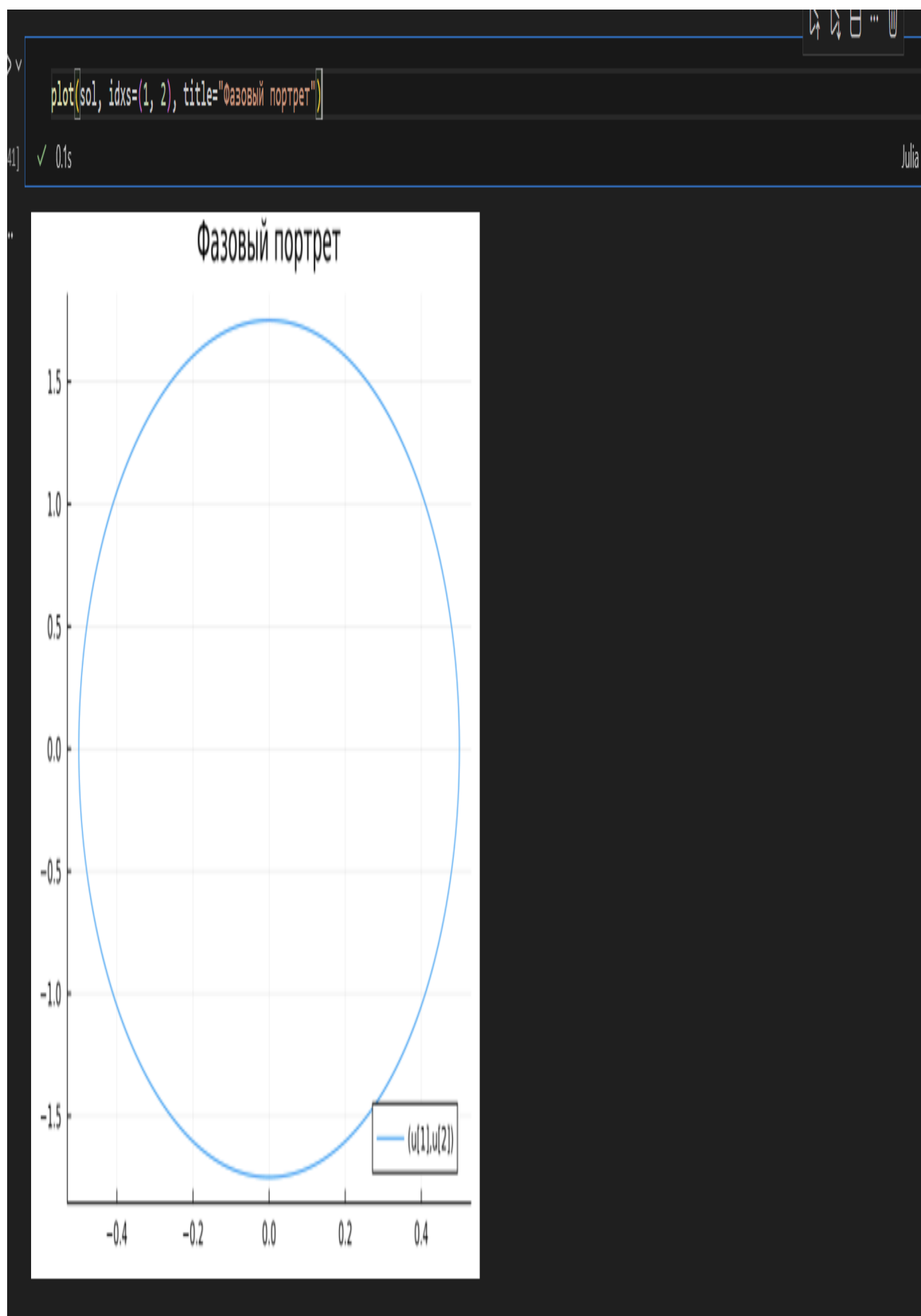


Рисунок 2.25: Решение задания №7

Выполнение задания №8 (рис. [fig:026] - рис. [fig:027]):

8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
tspan = (0.0, 5*pi)
u0 = [-0.5, 1]
p = [0.4, 3.5]
prob = ODEProblem(harmonic, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, title="Модель свободных колебаний гармонического осциллятора")
```

42] ✓ 0.0s

Julia

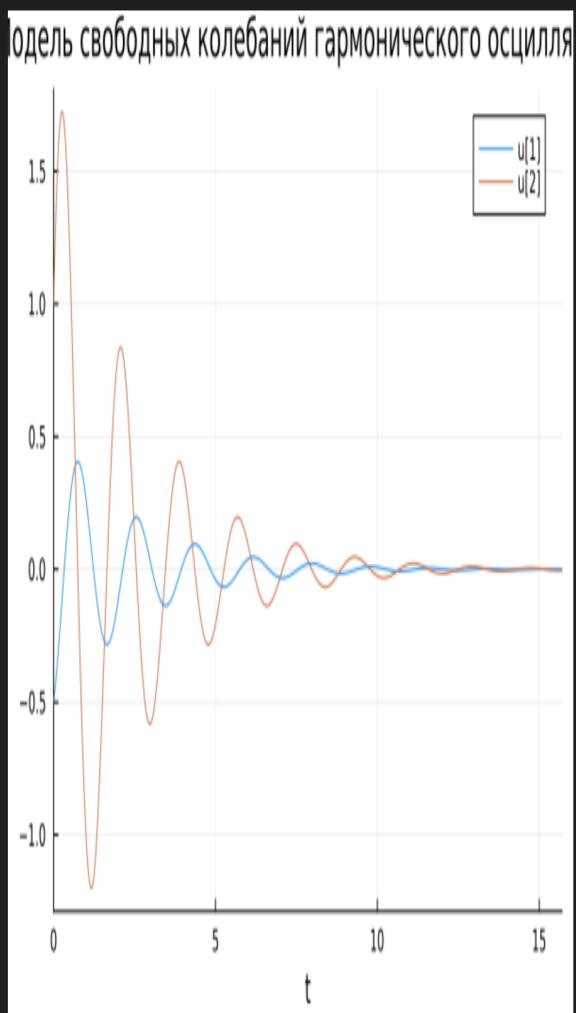


Рисунок 2.26: Решение задания №8



```
plot(sol, vars=(1, 2), title="Фазовый портрет")
```

[43]

✓ 0.0s

Julia

...

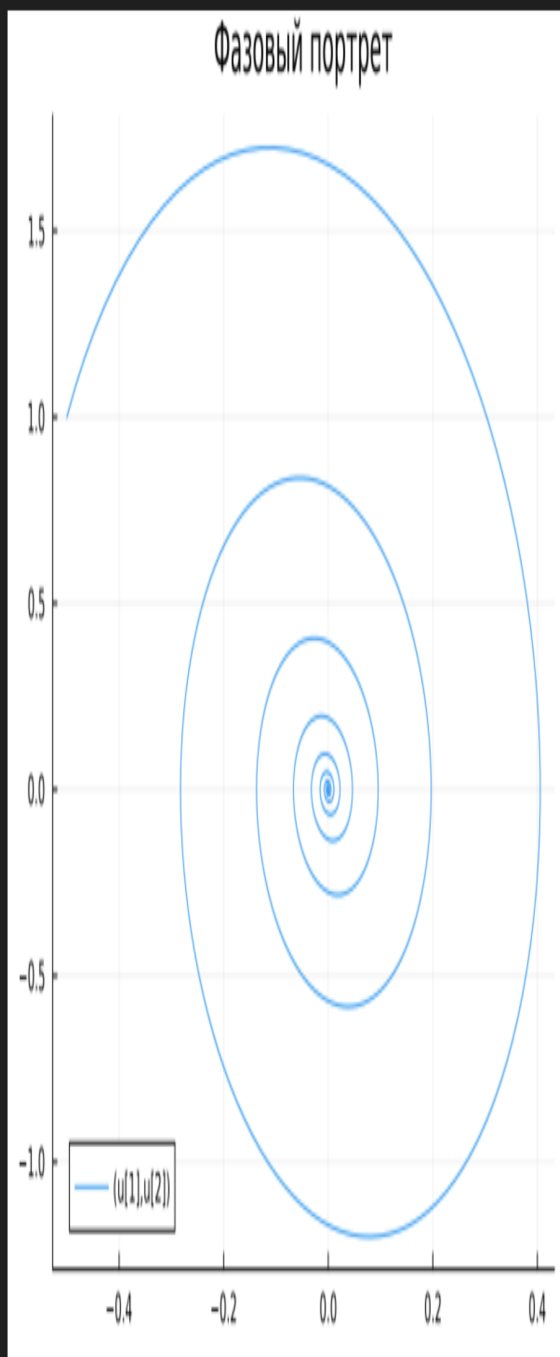


Рисунок 2.27: Решение задания №8

3 Выводы

В результате выполнения данной лабораторной работы я освоил специализированные пакеты для решения задач в непрерывном и дискретном времени.

Список литературы

- [1] *Julia 1.11 Documentation*. URL: <https://docs.julialang.org/en/v1/>.
- [2] *JuliaLang*. URL: <https://julialang.org/>.